

Engenharia de Software

Faculdade Jorge Amado
Prof. David Viscarra

Qualidade de Software

Qualidade de Software...

Nesta aula, abordaremos os conceitos fundamentais de Qualidade de Software, incluindo :

- definições,
- modelos,
- métricas e
- processos de garantia de qualidade.

Qualidade de Software...

Sua
opinião



Qualidade de Software...

- A qualidade de software é essencial para garantir que os produtos desenvolvidos sejam confiáveis, eficientes e atendam às necessidades dos usuários.

O que é Qualidade de Software?...

- De acordo com Roger Pressman, **qualidade de software** pode ser definida como a **conformidade** com requisitos funcionais e não funcionais, **garantindo** que o produto seja adequado ao uso.

Podemos diferenciar:

- **Qualidade de Produto:** refere-se às características do software em si.
 - **Qualidade de Processo:** foca nas metodologias utilizadas para desenvolver software de qualidade.
- 
- A blue bracket on the right side of the slide, grouping the two bullet points under the heading 'Podemos diferenciar:'.

A qualidade é **fundamental** para evitar falhas e aumentar a satisfação do cliente.

Dimensões da Qualidade de Software

A norma ISO/IEC 25010 define as características de qualidade:

- **Funcionalidade:** O software realiza as funções corretamente?
- **Confiabilidade:** O software funciona sem falhas?
- **Usabilidade:** É fácil de usar?
- **Eficiência:** Tem bom desempenho e consumo de recursos?
- **Manutenibilidade:** Pode ser atualizado facilmente?
- **Portabilidade:** Pode ser usado em diferentes plataformas?

Cada uma dessas dimensões impacta diretamente na satisfação do usuário e no sucesso do produto.

Modelos de Qualidade de Software

Os principais **modelos** de qualidade são:

- Modelo de **McCall**: Enfatiza requisitos internos e externos.
- Modelo de **Boehm**: Foca na utilidade para diferentes stakeholders.
- ISO 9126 (substituído pelo **ISO 25010**): Define padrões internacionais para qualidade.

Esses modelos ajudam empresas a estruturar e medir a qualidade de software de forma sistemática.

Garantia da Qualidade de Software (SQA)

A Garantia da Qualidade de Software (SQA) é um conjunto de atividades planejadas para garantir que os produtos de software atendam aos requisitos estabelecidos.

Principais práticas de SQA (Software Quality Assurance):

- Revisões técnicas formais (FTRs)
- Auditorias de qualidade
- Processos de melhoria contínua
- Automação de testes

A SQA é **essencial** para reduzir defeitos e melhorar a confiabilidade dos produtos.

Normas e Certificações de Qualidade

Normas :

ISO 9001: Conjunto de padrões internacionais para sistemas de gestão da qualidade, garantindo eficiência e melhoria contínua.

ISO 27001: Norma focada na segurança da informação, garantindo a proteção de dados sensíveis.

CMMI (Capability Maturity Model Integration): Modelo de maturidade que avalia e melhora processos de desenvolvimento de software.

ISO/IEC 25010: Define um modelo de qualidade para avaliação de software.

ISTQB (International Software Testing Qualifications Board): Certificação voltada para profissionais de testes de software.

Controle de Qualidade software

Definição :

O Controle de Qualidade (QC - Quality Control) é um conjunto de técnicas e atividades utilizadas para detectar e corrigir defeitos no software antes da entrega final.

Diferente da Garantia da Qualidade (QA), que foca na prevenção de defeitos, o QC tem um papel corretivo, identificando falhas nos produtos desenvolvidos.

Técnicas asseguram Qualidade software

Técnicas:

- Testes Funcionais.
- Testes de Regressão.
- Testes de Desempenho.
- Testes de Segurança.
- Revisão de Código.

→ Os testes de software são essenciais para garantir a qualidade ao **identificar falhas** antes da entrega do produto final.

→ A execução de testes eficazes **reduz o risco** de defeitos críticos e melhora a confiabilidade do software.

→ A **adoção de estratégias** de testes bem definidas ajuda a garantir que o software atenda aos requisitos funcionais e não funcionais.

Maturidade de Processos de Qualidade

O **CMMI** (Capability Maturity Model Integration) é um modelo de referência que visa melhorar os processos de desenvolvimento e manutenção de software

O modelo CMMI é dividido em cinco níveis de maturidade:

Inicial: Processos são ad hoc e caóticos. O sucesso depende de esforços individuais.

Gerenciado: Processos são projetados, planejados e monitorados.

Definido: Processos são padronizados e melhorados continuamente.

Quantitativamente Gerenciado: Processos são controlados usando medidas quantitativas.

Em Otimização: O foco está na melhoria contínua dos processos, com base em análises quantitativas.

Engenharia de Requisitos e Qualidade

Importância dos requisitos :

Definem **o que** o software deve fazer e são essenciais para garantir que o produto final atenda às necessidades dos usuários e stakeholders.

Fatores a considerar:

Requisitos claros e bem definidos

fornecem um guia para a equipe de desenvolvimento,
resultam em sistemas mais robustos

Fáceis de manter
ajudando a reduzir ambiguidades e
falhas durante o ciclo de vida do software.

Garantir a qualidade

O alinhamento entre os requisitos e as expectativas dos stakeholders é crucial para a satisfação do cliente.

Documentação de requisitos

clara e precisa,
incluir requisitos funcionais e não funcionais,
atender às expectativas de desempenho, segurança, usabilidade e outras métricas importantes.

Impacto em requisitos mal definidos

- **Desalinhamento com as necessidades do usuário**
- **Aumento de custos e atrasos**
- **Defeitos no software**
- **Dificuldades em testes e validação**
- **Falta de rastreabilidade**

Qualidade e Manutenção de Software

Relação entre qualidade e manutenibilidade:

- **Manutenibilidade** refere-se à facilidade com que o software pode ser modificado após sua entrega, seja para corrigir defeitos, adicionar novas funcionalidades ou melhorar o desempenho.
- **Qualidade de software e manutenibilidade** estão intimamente relacionadas. Um software de alta qualidade tende a ser mais fácil de manter, já que o código é mais claro, organizado e modular. Isso facilita a implementação de mudanças e a correção de erros com menor risco de introduzir novos problemas.
- A **manutenibilidade** é uma característica crucial para a longevidade do software. Sistemas que não são fáceis de manter acabam se tornando obsoletos rapidamente.
- Para garantir a manutenibilidade, é fundamental que o software seja bem projetado, modular e documentado.
- **Impacto no ciclo de vida:** a maioria dos custos de TI está associada à manutenção, e não ao desenvolvimento inicial.

Qualidade e Manutenção de Software

Técnicas de refatoração para manter a qualidade:

- Renomear variáveis e métodos
- Extrair métodos
- Remover código duplicado
- Reorganizar condicionais
- Substituir herança por composição (orientação objetos)
- Mover métodos entre classes (OO)

Benefícios refatoração

- Código mais legível, modular e eficiente;
- É mais fácil identificar e corrigir problemas;
- Equipes desenvolvimento trabalham mais rapidamente;
- Novos recursos sejam adicionados com menor risco de introduzir erros

Medição da Qualidade de Software

Métricas de qualidade:

- São ferramentas fundamentais para
 - Monitorar;
 - Medir e
 - Avaliar o desempenho do software

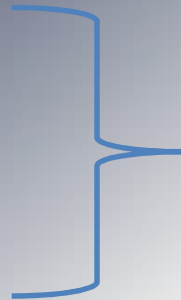
Exemplos Métricas

- **Cobertura de Testes** -- > Mede a porcentagem de código que foi testada
- **Tempo de Resposta** -- > Mede o tempo que o sistema leva para responder a uma solicitação;
- **Taxa de Defeitos** -- > quantidade de defeitos identificados durante o ciclo de vida do software;
- **Densidade de Defeitos** -- > número de defeitos por unidade de código;
- **Tempo de Ciclo** -- > O tempo necessário desde o planejamento até a entrega;
- **Taxa de Mudanças** -- > Mede a frequência de modificações

Qualidade e Experiência do Usuário (UX)

Experiência Usuário:

- O desempenho,
- a confiabilidade,
- a usabilidade e
- a acessibilidade



elementos essenciais
que garantem usuário
tenha uma experiência positiva

Fatores que influenciam:

- **Desempenho** -- > Software que demora afeta diretamente a satisfação do usuário;
- **Usabilidade** -- > interface intuitiva e simples, garante uma experiência boa;
- **Acessibilidade** -- > inclusiva e abrangente (deficiência visual, auditiva, etc.) ;
- **Segurança** -- > oferecer privacidade e segurança;

DevOps e Qualidade de Software

DecOps:

Abordagem **integra** equipes de **desenvolvimento (Dev)** e **operações (Ops)**

Objetivo melhorar a colaboração e aumentar a eficiência do ciclo de vida do software.

O **foco** principal é aprimorar as entregas através da automação e integração contínua

Resultado é garantir que os problemas de qualidade sejam identificados e corrigidos rapidamente.

Benefícios integração

- **Feedback mais rápido** -- > S falhas são detectadas cedo;
- **Cultura de responsabilidade compartilhada** -- > todos os membros compartilham a responsabilidade;
- **Redução de falhas em produção** -- > falhas em produção é reduzida, pois existem testes contínuos;

Como garantir Qualidade em Métodos Ágeis

Métodos Ágeis:

metodologias ágeis como Scrum, Kanban e Lean a **qualidade** é incorporada em todo o processo de desenvolvimento;

Entrega incremental: entrega de incrementos do produto em **ciclos curtos** (sprints);

Testes contínuos: constantemente **durante** o desenvolvimento;

Feedback rápido e constante: trabalho contínuo com o cliente e as **revisões** regulares;

Integração com o cliente: colaboração **constante** com o cliente ou usuário final;

Definição de "Pronto": (DoD - Definition of Done) critérios mínimos que precisam ser atendidos antes que uma tarefa ou história seja considerada concluída.

Benefícios



- Entrega rápida e contínua;
- Redução de riscos;
- Ajustes rápidos;

Dificuldades e estratégias para Garantir a Qualidade

Dificuldades:

- Falta de comunicação eficaz entre equipes;
- Gestão de requisitos e mudanças constantes;
- Testes inadequados ou mal planejados;
- Escalabilidade e desempenho;
- Pressão para entregar rapidamente;
- Desafios com integração contínua e automação;

Dificuldades e estratégias para Garantir a Qualidade

Estratégias:

Melhoria na comunicação e colaboração entre equipes;

- **Reuniões regulares:** Realizar **daily standups**, **reviews** e **retrospectives** com frequência;
- Utilizar ferramentas como Slack, Microsoft Teams ou outras plataformas;
- **Definir claramente os papéis e responsabilidades;**

Gestão eficiente de requisitos e mudanças;

- Definir claramente os requisitos iniciais;
- Utilizar histórias de usuário detalhadas e claras;
- Utilizar ferramentas de gestão de projetos como Jira ou Trello para acompanhar as mudanças

Planejamento adequado de testes e automação;

- Integrar **testes contínuos** ao longo de todo o ciclo de vida;
- Automatizar os testes repetitivos;
- Implementar **frameworks de testes** como Selenium, JUnit ou TestNG

Dificuldades e estratégias para Garantir a Qualidade

Estratégias:

Gestão de escalabilidade e desempenho

- Testes de carga e estresse;
- Implementar ferramentas de monitoramento como New Relic, Prometheus ou Grafana para detectar problemas de desempenho em tempo real;
- Refatoração do código;

Lidar com a pressão de entregas rápidas

- Priorização clara das funcionalidades;
- MVP (Produto Mínimo Viável);
- Automação de build e deploy;
- Equilibrar velocidade e qualidade;

Superando desafios com integração contínua e automação

- Investir em treinamento para a equipe de desenvolvimento sobre as melhores práticas;
- Investir em ferramentas de integração contínua como Jenkins, GitLab CI ou CircleCI;
- Testar o pipeline de CI/CD regularmente.

Casos Reais de Falha na Qualidade

Exemplos:

Therac-25 (1985): Um sistema de radiação utilizado para tratar pacientes com câncer apresentou falhas devido a erros de software que permitiram a administração de doses letais de radiação. O incidente resultou em pelo menos 3 mortes e várias lesões graves. Essa falha destacou a importância crítica de testar e validar software em sistemas de saúde.

Ariane 5 Rocket (1996): O foguete Ariane 5, lançado pela Agência Espacial Europeia, explodiu pouco após o lançamento devido a um erro de software. O sistema de navegação do foguete tentou converter valores de um tipo de dado para outro sem verificar a validade dos dados. O impacto financeiro foi significativo, com o custo do foguete e da missão perdidos, cerca de US\$ 370 milhões.

O Caso do Bug do Milênio (2000): O erro em muitos sistemas de software causado pela forma como os anos eram representados com dois dígitos (ex: 99 ao invés de 1999) levou a falhas em bancos de dados e sistemas financeiros, causando incertezas e erros no processamento de dados. Empresas gastaram bilhões de dólares para corrigir falhas antes e depois da virada do milênio.

Casos Reais de Falha na Qualidade

Exemplos:

Falhas de Software no Sistema de Controle de Trens em Londres (2013): Um erro em um sistema de controle de tráfego ferroviário fez com que diversos trens ficassem descoordenados, causando atrasos significativos e uma falha no serviço de transporte público. O impacto foi principalmente no serviço de transporte, mas também afetou a confiança do público na infraestrutura ferroviária.

Vírus Stuxnet (2010): Stuxnet foi um malware altamente sofisticado, desenvolvido para sabotagem de instalações nucleares no Irã. Sua presença mostrou como falhas de segurança em software industrial podem ter consequências geopolíticas graves e afetar operações críticas de infraestrutura.

Qualidade de Software no Futuro

Como a Inteligência Artificial (IA) impacta a qualidade de software:

- Automação de testes;
- Análise preditiva de qualidade;
- Detecção de anomalias em tempo real;
- Refatoração inteligente;
- Análise de requisitos;

Qualidade de Software no Futuro

Como novas tecnologias impactam a qualidade de software:

Blockchain: Ao incorporar blockchain para garantir a integridade dos dados, a segurança e a rastreabilidade das transações, software desenvolvido com essa tecnologia pode garantir maior confiança e resistência a falhas ou ataques. No entanto, a implementação correta de blockchain exige altos padrões de qualidade.

Computação em Nuvem: A nuvem oferece maior escalabilidade e flexibilidade, mas também introduz desafios em termos de segurança, integração e monitoramento. Ferramentas de qualidade de software para ambientes em nuvem precisam garantir a continuidade da performance e a conformidade com as políticas de segurança.

Qualidade de Software no Futuro

Como novas tecnologias impactam a qualidade de software:

Internet das Coisas (IoT): O crescimento do IoT aumenta a complexidade do desenvolvimento de software devido à integração de múltiplos dispositivos e plataformas. A qualidade deve ser garantida no desenvolvimento de sistemas que conectam esses dispositivos, com ênfase na segurança, confiabilidade e interoperabilidade.

Realidade Aumentada (AR) e Realidade Virtual (VR): As tecnologias AR e VR exigem que o software seja extremamente confiável, responsivo e de alta performance. Isso coloca uma pressão extra sobre as práticas de teste e desenvolvimento, exigindo testes mais rigorosos e ferramentas especializadas para garantir a qualidade.

Dúvidas e Perguntas

Momento para perguntas e esclarecimento de dúvidas.

Reflexão final sobre a importância da qualidade de software

Bibliografia

KOSCIANSKI, André; SOARES, Michel dos Santos. **Qualidade de Software**. Rio de Janeiro: Elsevier, 2007.

BARTIÉ, Alexandre. **Garantia da Qualidade de Software: Adquirindo Maturidade Organizacional**. São Paulo: Novatec, 2010.

GALLOTTI, Giocondo Marino Antonio. **Qualidade de Software**. São Paulo: Erica, 2014.
SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

DEVMEDIA. **Qualidade de Software: Conceitos e Características**. 2022. Disponível em: <https://www.devmedia.com.br/qualidade-de-software-engenharia-de-software-29/18209>. Acesso em: 27 mar. 2025.

BEECROWD. **Qualidade de Software e a sua importância em projetos**. 2022. Disponível em: <https://beecrowd.com/pt/blog-posts/software-de-qualidade/>. Acesso em: 27 mar. 2025.

WIKIPÉDIA. **ISO/IEC 9126**. Disponível em: https://pt.wikipedia.org/wiki/ISO/IEC_9126. Acesso em: 27 mar. 2025.